



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Alternate Assessment Tool

Course Title: INTERNET OF THINGS AND 5G

Course Code: 22AI642

Semester: VI

Submitted by

Ritvik K – 1DS23AI045

Under the Guidance of

Dr Aruna M G

Professor

Department of Artificial Intelligence & Machine Learning

DSCE, Bengaluru

Visvesvaraya Technological University

Jnanasangama, Belagavi-590018, Karnataka, India

2025-26

Table of Contents

Activity 1: Tinker CAD Aurdino.....	1
Activity 2: Wokwi ESP32	7
Activity 3: Wokwi STM.....	13
Activity 4: Wokwi Raspberry Pi Pico simulator.....	18
Activity 5 : TinyML simulation in Wokwi.....	23

Activity 1: Tinker CAD Aurdino

Problem Statement	Inefficient waste management often leads to overflowing dustbins, unhygienic conditions, and delayed collection due to the lack of real-time monitoring. There is a need for an automated system that can detect and report the fill level of bins.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	• Tinker CAD • Arduino Uno Microcontroller • Ultrasonic Sensor (HC-SR04 / PING))) • Jumper Wires (Male-to-Male) • USB Cable (for power supply)				
MAP SDG	3.9: Reduce illnesses caused by pollution and contamination. 11.6: Reduce the adverse environmental impact of cities by improving waste management systems. 12.5: Substantially reduce waste generation through prevention, reduction, recycling, and reuse.				
MAP COURSE OUTCOME of IOT	CO	Mapping	Justification		
	CO1	Yes	Demonstrates IoT perception layer using ultrasonic sensor to detect waste level and microcontroller processes the data		
	CO2	Yes	Uses Arduino programming and sensor interfacing to monitor and control dustbin status		
	CO3	No	No advanced data analytics or cloud-based analysis used in this basic implementation		
	CO4	Yes	Uses ultrasonic sensor (sensing) and buzzer/LED (actuation) forming a complete sense-process-act IoT system		
MAP PROGRAM OUTCOME	PO	Mapping	Justification		

	PO1	Yes	Applied knowledge of electronics (ultrasonic sensor), programming, and embedded systems
	PO2	Yes	Identified real-world problem of inefficient waste management and analyzed bin fill levels
	PO3	Yes	Designed an automated smart dustbin system ensuring hygiene and timely waste collection
	PO5	Yes	Used TinkerCAD simulation tool and Arduino IDE for development and testing
	PO6	Yes	Addresses societal and environmental issues like cleanliness, public health, and sustainability (SDG 3, 11, 12)
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO1	Yes	Demonstrates programming and problem-solving using Arduino code for sensor data processing and alerts
	PSO2	Yes	Solves real-world waste management problem using IoT and embedded system concepts
	PSO3	Yes	Uses TinkerCAD as a computational platform to simulate and deploy the IoT solution
URL Link	https://www.tinkercad.com/things/fnHBLgTfZSf-smartdustbin-ritvik/editel?returnTo=%2Fdashboard%2Fdesigns%2Fall&sharecode=7SdDQoHk0sj6rUMawRZqggxELKMxnA8pRVPJhxUD2yc		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre> classDiagram class SmartDustbin { -ledPin : int = 7 -threshold : int = 20 -status : String +setup() : void +loop() : void } class UltrasonicSensor { -trigPin : int -echoPin : int -rawValue : float +readDistance() : int +simulate() : int } class LEDIndicator { -pin : int -state : bool +turnOn() : void +turnOff() : void +getState() : bool } class SerialMonitor { -baudRate : int -lastOutput : String +printStatus() : void +logDistance() : void +begin() : void } SmartDustbin ..> UltrasonicSensor : uses SmartDustbin ..> LEDIndicator : uses SmartDustbin ..> SerialMonitor : uses </pre> Class diagram — Smart Dustbin IoT System		

Details**Abstract :**

The Smart Dustbin is an IoT-based project designed to monitor the level of waste inside a dustbin using an ultrasonic sensor. The system detects the distance between the garbage and the sensor to determine whether the bin is full or empty. Based on this data, it displays the status in the serial monitor and can trigger alerts or indicators. This helps in maintaining cleanliness and efficient waste management.

Use Case:

This system can be used in public places such as streets, parks, schools, and hospitals to monitor garbage levels. It helps municipal workers to identify when bins are full and need to be emptied, reducing overflow and improving hygiene.

Introduction:

With the increasing population, waste management has become a major challenge. Traditional dustbins require manual checking, which is inefficient. The Smart Dustbin uses IoT technology and sensors to automate this process. It provides real-time information about the bin status, making waste collection more effective and systematic.

Objective:

- To design an automated waste monitoring system
- To detect the garbage level using a sensor
- To indicate whether the bin is full or empty
- To improve cleanliness and reduce manual effort

General procedure

- Connect the ultrasonic sensor to the Arduino board

-
- Provide power supply to the circuit
 - Upload the program (code) to the Arduino
 - Start the simulation
 - Observe the distance values and bin status in the serial monitor

Step to Execute:

- Open the simulation platform (Tinkercad)
- Place Arduino Uno and ultrasonic sensor
- Connect VCC, GND, Trigger, and Echo pins properly
- Write or paste the code in the editor
- Click on “Start Simulation”
- Open Serial Monitor to view output
- Change object distance to test bin full/empty condition

Source Code:

```
int led = 7;

void setup() {
  Serial.begin(9600);
  randomSeed(analogRead(0)); //
  pinMode(led, OUTPUT);
}

void loop() {
  int distance = random(5, 100); // distance between 5–100 cm
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  // Bin logic
  if (distance < 20) {
    Serial.println(" Bin Full");
    digitalWrite(led, HIGH);
  } else {
    Serial.println(" Bin Empty");
  }
}
```

```
digitalWrite(led, LOW);}

Serial.println("-----");

delay(1000);}
```

Output:



```
Serial Monitor

Distance: 46 cm
Bin Empty

-----

Distance: 47 cm
Bin Empty

-----

Distance: 22 cm
Bin Empty

-----

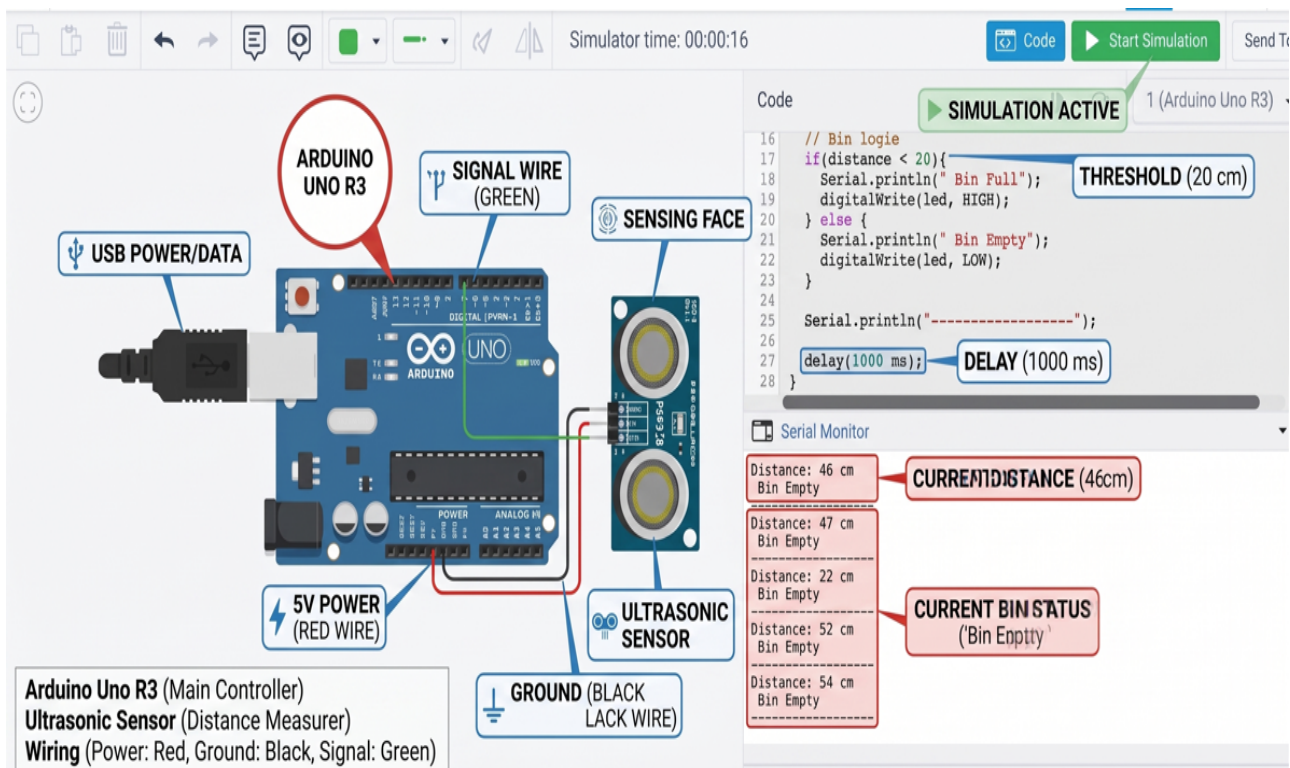
Distance: 25 cm
Bin Empty

-----

Distance: 54 cm
Bin Empty

-----
```

Photos/ Screenshot/Snapshot:

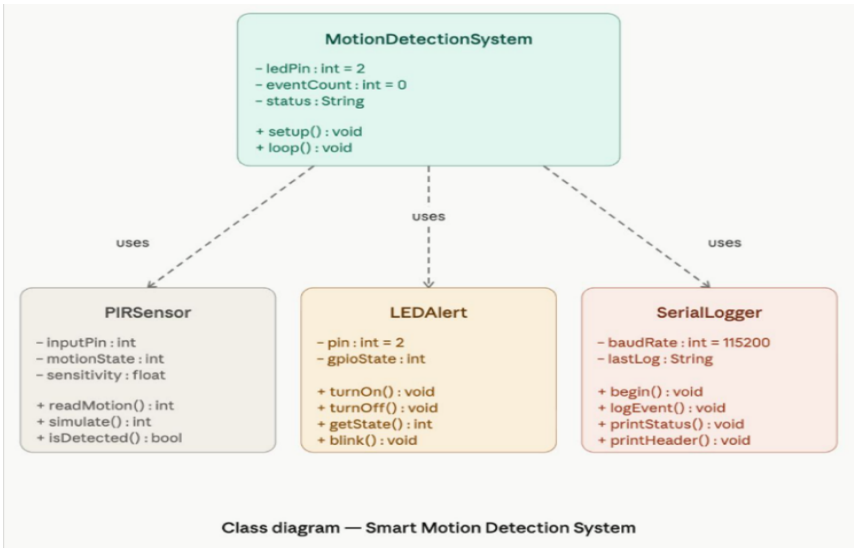


Outcome:

The Smart Dustbin system successfully detects the garbage level using the ultrasonic sensor. It displays whether the bin is full or empty based on distance values. This demonstrates how IoT can be used for efficient waste management and cleanliness maintenance.

Activity 2: Wokwi ESP32

Problem Statement	Security is a major concern in homes and industries. There is a need for a smart system that can detect human motion and trigger alerts automatically.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	• Wokwi Simulator – Online IoT Simulation Platform • ESP32 DevKit V1 Microcontroller • PIR Motion Sensor (HC-SR501) • LED (Red) • Jumper Wires (Male-to-Male) • Serial Monitor (for output display in Wokwi)				
MAP SDG	9.1: Develop reliable, sustainable, and resilient infrastructure to support economic development and human well-being. 11.7: Provide safe, inclusive, and accessible public and residential spaces.				
MAP COURSE OUTCOME of IOT/ BFE	CO	Mapping	Justification		
	CO1	Yes	Demonstrates IoT perception layer using PIR sensor to detect human motion and microcontroller processes the signal		
	CO2	Yes	Uses Arduino programming and digital input from PIR sensor with serial/alert mechanisms		
	CO3	No	No data analytics or cloud-based processing involved in this standalone system		
	CO4	Yes	Uses PIR sensor (sensing) and buzzer/LED (actuation) forming a complete sense-process-act IoT loop		
MAP PROGRAM OUTCOME					
	PO	Mapping	Justification		
	PO1	Yes	Applied knowledge of electronics (PIR sensor), embedded systems, and programming concepts		

	PO2	Yes	Identified and analyzed real-world security problem using motion detection logic
	PO3	Yes	Designed an automated security system that detects intrusion and triggers alerts
	PO5	Yes	Used Wokwi simulation platform and Arduino IDE for development and testing
	PO6	Yes	Contributes to safety and security in homes and industries (aligned with SDG 9 and SDG 11)
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO1	Yes	Demonstrates programming and logical implementation using Arduino code for motion detection and alert system
	PSO2	Yes	Solves real-world security problem using IoT-based sensing and automation
	PSO3	Yes	Uses Wokwi simulation platform to deploy and test the IoT-based security system
URL Link	https://wokwi.com/projects/460845508505291777		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model			
COURSE COORDINATOR	Dr ARUNA M G		

Details
Abstract: This project presents a Smart Motion Detection System using ESP32 and a PIR sensor. The system detects human motion based on infrared radiation and triggers an LED alert. It provides a simple and efficient solution for security applications. The system can be extended to include alarms or notifications for real-time monitoring.
Use Case: Home security systems and office monitoring solutions use smart surveillance systems to ensure safety and continuous oversight of premises. These systems help detect intrusions, monitor activities, and provide real-time alerts. They are especially useful for restricted area monitoring, ensuring only authorized access and enhancing overall security.
Introduction: Motion detection plays a crucial role in modern security systems. PIR (Passive Infrared) sensors detect infrared radiation emitted by human bodies. When motion is detected, the ESP32 processes the signal and activates an alert system. This project demonstrates a simple, cost-effective, and efficient method for real-time motion detection using IoT technology.
Objective: • To detect human motion using a PIR sensor • To process sensor data using ESP32 • To trigger an alert using LED • To simulate real-time monitoring system
General procedure • Connect PIR sensor to ESP32 input pin • Connect LED to ESP32 output pin • Initialize the system and sensor • Continuously read PIR sensor output • Detect motion signal

-
- Turn ON/OFF LED based on detection

Step to Execute:

- Open Wokwi ESP32 simulator
- Add ESP32, PIR sensor, and LED
- Connect all components properly
- Paste the code in the editor
- Start the simulation
- Toggle the PIR sensor to simulate motion
- Observe LED response

Source Code:

```
int led = 2;
int eventCount = 0;
void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  randomSeed(analogRead(0));
  Serial.println("=====");
  Serial.println(" SMART MOTION DETECTION SYSTEM ");
  Serial.println(" ESP32 Simulation Started ");
  Serial.println("=====");
}
void loop() {
  int motion = random(0, 2); // simulate motion (0 or 1)
  eventCount++;
  Serial.println("\n-----");
  Serial.print("Event #: ");
  Serial.println(eventCount);
  if(motion == 1){
    Serial.println("Status    : Motion Detected");
    Serial.println("Action   : LED ON");
```

```
digitalWrite(led, HIGH);} else {
Serial.println("Status   : No Motion");
Serial.println("Action    : LED OFF"); digitalWrite(led, LOW);}
Serial.print("GPIO State : ");
Serial.println(digitalRead(led));Serial.println("-----");delay(1000);}
```

Output:

```
Status      : No Motion
Action      : LED OFF
GPIO State  : 0

-----

Event #: 7
Status      : Motion Detected
Action      : LED ON
GPIO State  : 1

-----

Event #: 8
```

Photos/ Screenshot/Snapshot:

The screenshot displays the WOKWI simulation interface. On the left, the 'sketch.ino' file contains the following code:

```
1 int led = 2;
2 int eventCount = 0;
3
4 void setup() {
5   Serial.begin(115200);
6   pinMode(led, OUTPUT);
7   randomSeed(AnalogRead(0));
8
9   Serial.println("-----");
10  Serial.println(" SMART MOTION DETECTION SYSTEM ");
11  Serial.println(" ESP32 Simulation Started ");
12  Serial.println("-----");
13 }
14
15 void loop() {
16   int motion = random(0, 2); // simulate motion (0 or 1)
17   eventCount++;
18
19   Serial.println("\n-----");
20   Serial.print("Event #: ");
21   Serial.println(eventCount);
22
23   if(motion == 1){
24     Serial.println("Status   : Motion Detected");
25     Serial.println("Action    : LED OFF");
26     digitalWrite(led, HIGH);
27   } else {
```

Annotations on the code include 'LED Pin Assignment' pointing to line 1 and a legend at the bottom left:

- ESP32 Board: Runs code, controls peripherals
- PIR Sensor: Detects motion
- Red LED: Visual signal
- Wire: Electrical connection

On the right, the 'Simulation' window shows a hardware diagram with labels: 'PIR Motion Sensor (Motion Input)', 'LED Indicator (Pin D2 Output)', 'ESP32 Dev Board (Processor)', and 'GPIO 2 (Red LED Wire)'. Below the diagram is the 'Current Simulation Event' log:

```
Event #: 7
Status      : Motion Detected
Action      : LED ON
GPIO State  : 1
```

An annotation 'Action Output (matches LED state)' points to the 'LED ON' text in the event log.

Outcome:

The Smart Motion Detection System successfully detects human motion and activates an LED alert. The system demonstrates real-time monitoring and provides a simple and effective solution for security applications using IoT technology.

Activity 3: Wokwi STM

Problem Statement	Traffic congestion and poor signal coordination are major issues in urban areas. Manual or poorly timed traffic signals can lead to delays, accidents, and inefficient traffic flow. There is a need for an automated system that can control traffic lights.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	• Wokwi Simulator – Online Embedded System Simulation Platform • STM Nucleo C031C6 Microcontroller Board • Breadboard • 3 LEDs (Red, Yellow, Green) • Resistor (1kΩ) • Jumper Wires (Male-to-Male) • Serial Monitor (for debugging/output)				
MAP SDG	3.6: Reduce the number of global deaths and injuries from road traffic accidents. 9.1: Develop reliable, sustainable, and resilient infrastructure to support efficient transportation systems. 11.2: Provide access to safe, affordable, and sustainable transport systems for all.				
MAP COURSE OUTCOME of IOT/ BFE	CO	Mapping	Justification		
	CO1	Yes	Demonstrates IoT architecture where microcontroller controls traffic signals representing processing layer with timed logic		
	CO2	Yes	Uses embedded C/Arduino programming for controlling LEDs with timing sequences and signal coordination		
	CO3	No	No data analytics or real-time traffic data processing involved in this basic implementation		
	CO4	Yes	Uses LEDs as actuators controlled by programmed logic forming a process-act system (can be extended with sensors)		

MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of electronics (LED circuits), timing logic, and embedded programming
	PO2	Yes	Identified and analyzed real-world problem of traffic congestion due to poor signal timing
	PO3	Yes	Designed an automated traffic light system to improve traffic flow and reduce delays
	PO5	Yes	Used Wokwi simulation platform and embedded programming tools for implementation
	PO6	Yes	Contributes to efficient urban infrastructure and safer transportation (aligned with SDG 9 and SDG 11)
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO1	Yes	Demonstrates programming and problem-solving through traffic signal timing logic implementation
	PSO2	Yes	Solves real-world traffic management problem using embedded systems concepts
	PSO3	Yes	Uses Wokwi STM simulation platform to deploy and test the traffic control system
URL Link	https://wokwi.com/projects/460847031455323137		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre> classDiagram class TrafficLightSystem { -redPin : int = D2 -yellowPin : int = D3 -greenPin : int = D4 -currentSignal : String +setup() : void +loop() : void } class RedLight { -pin : int = D2 -duration : int = 2000 -state : bool +activate() : void +deactivate() : void +getDuration() : int } class YellowLight { -pin : int = D3 -duration : int = 1000 -state : bool +activate() : void +deactivate() : void +getDuration() : int } class GreenLight { -pin : int = D4 -duration : int = 2000 -state : bool +activate() : void +deactivate() : void +getDuration() : int } class SerialLogger { -baudRate : int = 115200 -lastLog : String +begin() : void +logSignal() : void +printHeader() : void +printStatus() : void } TrafficLightSystem --> RedLight : uses TrafficLightSystem --> YellowLight : uses TrafficLightSystem --> GreenLight : uses TrafficLightSystem --> SerialLogger : uses </pre> Class diagram — Smart Traffic Light Control System		
COURSE COORDINATOR	Dr ARUNA M G		

Details
Abstract: This project presents a Smart Traffic Light Control System using STM32. It simulates the working of traffic signals by controlling RED, YELLOW, and GREEN lights in a sequential manner. The system uses time-based logic to switch between signals, ensuring proper traffic flow and safety.
Use Case: Smart city traffic management leverages advanced systems for road intersection control, real-time monitoring, and traffic simulation to improve mobility and reduce congestion. These technologies support efficient urban infrastructure development by optimizing traffic flow and enhancing safety. Together, they enable smarter planning and more responsive city transportation networks.
Introduction: Traffic management is essential for maintaining order and safety on roads. Traditional systems may lack proper timing and coordination. This project demonstrates a simple automated traffic control system using STM32, where signals change based on fixed time intervals. It provides a basic understanding of how real-world traffic systems operate.
Objective: • To simulate traffic signal operation • To implement time-based control logic • To automate switching between signals • To improve traffic flow understanding
General procedure • Initialize STM32 system • Define sequence of traffic lights • Apply time delay for each signal • Display current signal status • Repeat the cycle continuously
Step to Execute:

-
- Set up STM32 simulation environment
 - Write or paste the C code
 - Compile and run the program
 - Observe traffic signal sequence
 - Verify correct timing and order
 - Repeat cycles for validation

Source Code:

```
int red = D2;
int yellow = D3;
int green = D4;
void setup() {
  pinMode(red, OUTPUT);
  pinMode(yellow, OUTPUT);
  pinMode(green, OUTPUT);
  Serial.begin(115200);
  Serial.println(" 🚦 Smart Traffic Light System Started");
}
void loop() {
  // RED
  Serial.println(" 🔴 RED Light - STOP");
  digitalWrite(red, HIGH);
  digitalWrite(yellow, LOW);
  digitalWrite(green, LOW);
  delay(2000);
  // YELLOW
  Serial.println(" 🟡 YELLOW Light - READY");
  digitalWrite(red, LOW);
  digitalWrite(yellow, HIGH);
  digitalWrite(green, LOW);
  delay(1000);
  // GREEN
```

```

Serial.println(" ● GREEN Light - GO");
digitalWrite(red, LOW);
digitalWrite(yellow, LOW);
digitalWrite(green, HIGH);
delay(2000);
}

```

Output:

```

Smart Traffic Light System Started
RED Light - STOP
YELLOW Light - READY
GREEN Light - GO
RED Light - STOP

```

Photos/ Screenshot/Snapshot:

LED Pin Definitions

Setup Function

Control Logic (RED)

Control Logic (YELLOW)

Control Logic (GREEN)

NUCLEO C031C6 Board

RED LED (Active)

YELLOW LED (Inactive)

GREEN LED (Inactive)

Current-Limiting Resistors

GND Connection (Common)

System Start Message

Current Status Message (RED Light)

Outcome:

The Smart Traffic Light Control System successfully simulates traffic signal operations using time-based control. It demonstrates how embedded systems can be used for efficient traffic management and smart city applications.

Activity 4: Wokwi Raspberry Pi Pico simulator

Problem Statement	Unauthorized access and lack of real-time monitoring are common security issues in homes and offices. Traditional systems may not provide immediate alerts when a door is opened.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	• Wokwi Simulator – Online IoT Simulation Platform • Raspberry Pi Pico Microcontroller • PIR Motion Sensor (HC-SR501) • LED (Red) • Buzzer • Breadboard • Jumper Wires (Male-to-Male)				
MAP SDG	9.1: Develop reliable, sustainable, and resilient infrastructure to support secure systems and human well-being. 11.7: Provide safe, inclusive, and accessible public and residential spaces.				
MAP COURSE OUTCOME of IOT/ BFE	CO	Mapping	Justification		
	CO1	Yes	Demonstrates IoT perception layer using magnetic/reed switch sensor to detect door status and microcontroller processes the input		
	CO2	Yes	Uses Arduino programming and digital input handling along with alert mechanisms like buzzer/LED		
	CO3	No	No data analytics or cloud-based monitoring implemented in this basic system		
	CO4	Yes	Uses door sensor (sensing) and buzzer/LED (actuation) forming a complete sense-process-act IoT loop		
MAP PROGRAM OUTCOME					

	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of electronics (door sensor), embedded systems, and programming fundamentals
	PO2	Yes	Identified and analyzed real-world problem of unauthorized access using sensor-based detection
	PO3	Yes	Designed an automated door monitoring system that triggers alerts for security enhancement
	PO5	Yes	Used Wokwi simulation platform and Arduino IDE for development and testing
	PO6	Yes	Improves safety and security in homes and offices (aligned with SDG 9 and SDG 11)
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO1	Yes	Demonstrates programming and problem-solving using Arduino code for door status detection and alert logic
	PSO2	Yes	Solves real-world security problem using IoT-based sensing and automation
	PSO3	Yes	Uses Wokwi simulation platform to deploy and test the IoT-based security solution
URL Link	https://wokwi.com/projects/460851393151686657		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre> classDiagram class DoorAlertSystem { - buttonPin : int = 14 - ledPin : int = 15 - buzzerPin : int = 16 - doorStatus : String + run() : void + checkDoor() : void } class DoorSensor { - pin : int = 14 - pullUp : bool = True - state : int + readState() : int + isOpen() : bool + simulate() : int } class LEDIndicator { - pin : int = 15 - brightness : int - state : bool + turnOn() : void + turnOff() : void + getState() : bool } class BuzzerAlert { - pin : int = 16 - frequency : int - active : bool + beep() : void + silence() : void + isActive() : bool } class SerialMonitor { - interval : float = 0.5 - lastMessage : String + printOpen() : void + printClosed() : void + logStatus() : void + sleep() : void } DoorAlertSystem ..> DoorSensor : uses DoorAlertSystem ..> LEDIndicator : uses DoorAlertSystem ..> BuzzerAlert : uses DoorAlertSystem ..> SerialMonitor : uses </pre> Class diagram — Smart Door Alert System		

COURSE COORDINATOR	Dr ARUNA M G
---------------------------	--------------

Details
Abstract: This project presents a Smart Door Alert System using Raspberry Pi Pico. The system detects whether a door is open or closed using a push button. When the door is opened, an alert is triggered using a buzzer and LED. This system provides a simple and effective solution for basic security monitoring.
Use Case: Home security systems and office access monitoring solutions use smart door alert systems to track entry and exit in real time. These technologies enhance safety by detecting unauthorized access and sending instant notifications. They are especially effective for restricted area monitoring, ensuring only approved individuals can enter sensitive spaces.
Introduction: Security is an important aspect of modern living. Monitoring door status helps in preventing unauthorized access. This project uses a push button to simulate a door and detects its state using Raspberry Pi Pico. When the door is opened, the system activates an alert using LED and buzzer, providing immediate feedback.
Objective: • To detect door open/close status • To trigger alert using buzzer • To indicate status using LED • To simulate real-time security system
General procedure • Connect push button to Pico input pin • Connect LED and buzzer to output pins • Initialize input and output devices

-
- Continuously read button state
 - Detect door open/close condition
 - Activate or deactivate alert system

Step to Execute:

- Set up Raspberry Pi Pico environment
- Connect button, LED, and buzzer properly
- Upload MicroPython code
- Run the program
- Press button to simulate door opening
- Observe LED and buzzer response
- Verify output messages

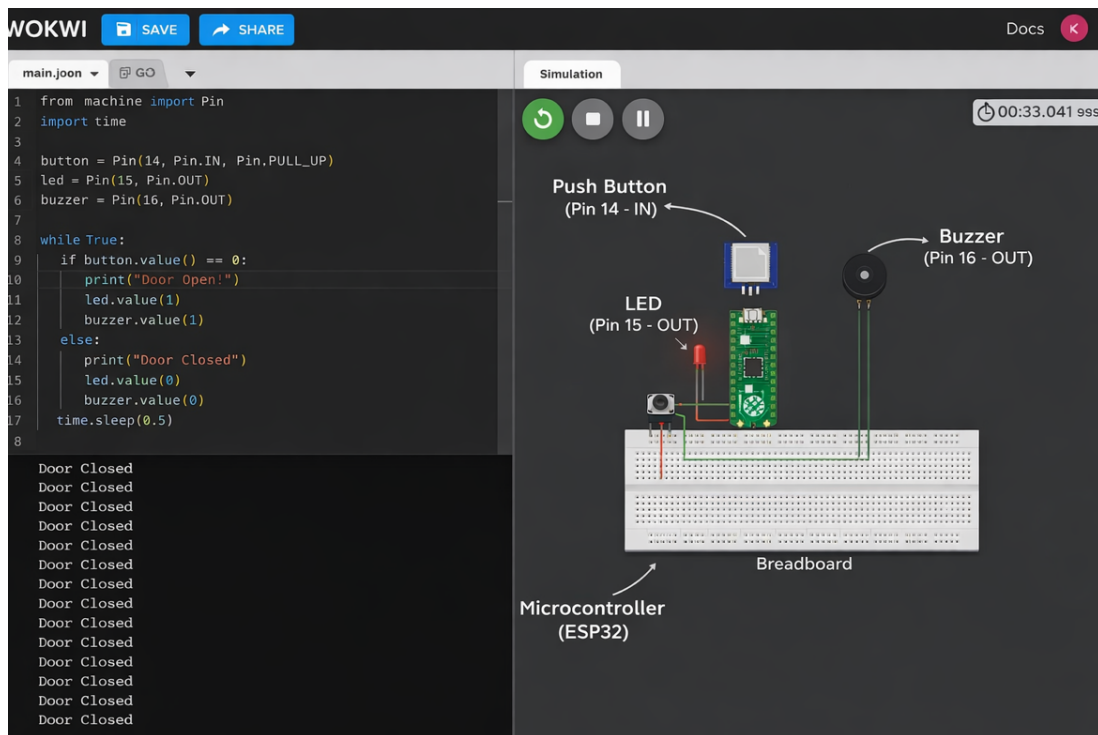
Source Code

```
from machine import Pin
import time
button = Pin(14, Pin.IN, Pin.PULL_UP)
led = Pin(15, Pin.OUT)
buzzer = Pin(16, Pin.OUT)
while True:
    if button.value() == 0:
        print(" Door Open!")
        led.value(1)
        buzzer.value(1)
    else:
        print("Door Closed")
        led.value(0)
        buzzer.value(0)
    time.sleep(0.5)
```

Output:

```
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
Door Closed
```

Photos/ Screenshot/Snapshot:



Outcome:

The Smart Door Alert System successfully detects door status and triggers alerts when the door is opened. It demonstrates a simple and effective IoT-based security solution using Raspberry Pi Pico.

Activity 5 : TinyML simulation in Wokwi

Problem Statement	In industrial environments, machine failures can lead to production loss, increased maintenance costs, and safety risks. Traditional maintenance methods may not detect faults early.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	• Wokwi Simulator – Online IoT Simulation Platform • ESP32 DevKit V1 Microcontroller • Potentiometer (Analog Input Sensor) • Jumper Wires (Male-to-Male) • Serial Monitor (for displaying analog values)				
MAP SDG	8.2: Achieve higher levels of economic productivity through technological upgrading and innovation. 9.1: Develop reliable, sustainable, and resilient infrastructure to support industrial growth and innovation.				
MAP COURSE OUTCOME of IOT/BFE	CO	Mapping	Justification		
	CO1	Yes	Demonstrates IoT perception layer using sensors (e.g., vibration/temperature) to detect machine condition and microcontroller processes data		
	CO2	Yes	Uses Arduino programming and sensor interfacing to monitor machine parameters and generate alerts		
	CO3	No	No advanced data analytics or predictive modeling implemented in this basic simulation		
	CO4	Yes	Uses sensors (sensing) and buzzer/LED (actuation) forming a complete sense-process-act IoT loop		
MAP PROGRAM OUTCOME	PO	Mapping	Justification		
	PO1	Yes	Applied knowledge of electronics (sensors), embedded systems, and programming fundamentals		

	PO2	Yes	Identified and analyzed real-world problem of machine failure using sensor-based monitoring
	PO3	Yes	Designed a fault detection system to provide early alerts and reduce downtime
	PO5	Yes	Used Wokwi simulation platform and Arduino tools for implementation and testing
	PO6	Yes	Supports industrial productivity and safety (aligned with SDG 9 and SDG 8)
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO1	Yes	Demonstrates programming and problem-solving using Arduino code for monitoring and alert logic
	PSO2	Yes	Solves real-world industrial problem using IoT and embedded system concepts
	PSO3	Yes	Uses Wokwi simulation platform to deploy and test the IoT-based fault detection system
URL Link	https://wokwi.com/projects/460936718525736961		
SYSTEM DIAGRAM / UML / SEQUENCE DIAGRAM/ 3D Model	<pre> classDiagram class FaultDetectionSystem { - sensorPin : int = 36 - threshold : float = 0.5 - status : String + setup() : void + loop() : void } class VibrationSensor { - pin : int - rawValue : float + readValue() : float + simulate() : float + normalize() : float } class MLClassifier { - threshold : float - model : String + classify() : String + loadModel() : void + predict() : bool } class AlertSystem { - baudRate : int - lastAlert : String + sendAlert() : void + printStatus() : void + logData() : void } FaultDetectionSystem ..> VibrationSensor : uses FaultDetectionSystem ..> MLClassifier : uses FaultDetectionSystem ..> AlertSystem : uses </pre>		
COURSE COORDINATOR	Dr ARUNA M G		

Details
Abstract : This project presents a Smart Machine Fault Detection System based on vibration analysis. It uses simulated vibration data to classify machine conditions as normal or faulty. A simple threshold-based or machine learning model is used to detect faults. This system can help industries perform predictive maintenance and reduce unexpected failures.
Use Case: Industrial machine monitoring and equipment health monitoring enable real-time tracking of machinery performance to detect anomalies early and ensure optimal operation. Predictive maintenance systems leverage this data to anticipate failures, reducing downtime and maintenance costs. Together with factory automation, these technologies improve efficiency, reliability, and overall productivity in industrial environments.
Introduction: Machine failures in industries can cause serious losses and safety issues. Monitoring machine health is essential for efficient operation. This project uses vibration data to detect faults in machines. By analyzing vibration patterns, the system can identify abnormal behavior and provide early warnings, making maintenance more efficient.
Objective: • To monitor machine vibration levels • To classify machine condition (normal/faulty) • To implement simple ML or threshold-based detection • To improve maintenance efficiency
General procedure • Collect vibration data (normal and faulty)

- Train classification model (optional using Edge Impulse)
- Deploy model to microcontroller
- Read vibration values (sensor/simulated)
- Analyze data using threshold or model
- Display machine status

Step to Execute:

- Create project in Edge Impulse (optional)
- Upload vibration data (normal and faulty)
- Train classification model
- Deploy model as Arduino library
- Open Wokwi and select ESP32
- Paste code and run simulation
- Observe output in Serial Monitor

Source Code:

```
#include "TensorFlowLite.h"
#include "model.h"
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "tensorflow/lite/version.h"
#define SENSOR_PIN 36
// TensorFlow globals
tflite::MicroInterpreter* interpreter;
TfLiteTensor* input;
TfLiteTensor* output;
// Memory buffer
constexpr int tensorArenaSize = 2 * 1024;
```

```
uint8_t tensorArena[tensorArenaSize];

void setup() {
    Serial.begin(115200);
    // Load model
    const tflite::Model* model = tflite::GetModel(model_tflite);
    static tflite::AllOpsResolver resolver;
    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensorArena, tensorArenaSize
    );
    interpreter = &static_interpreter;
    interpreter->AllocateTensors();
    input = interpreter->input(0);
    output = interpreter->output(0);
}

void loop() {
    int sensorValue = analogRead(SENSOR_PIN);
    float vibration = sensorValue / 4095.0;
    // Feed input to model
    input->data.f[0] = vibration;
    // Run inference
    interpreter->Invoke();
    float prediction = output->data.f[0];
    Serial.print("Vibration: ");
    Serial.print(vibration);
    Serial.print(" -> Prediction: ");
    Serial.print(prediction);
```

```
if (prediction > 0.5) {  
    Serial.println(" => Fault Detected!");  
} else {  
    Serial.println(" => Machine Normal");  
}  
delay(1000);  
}
```

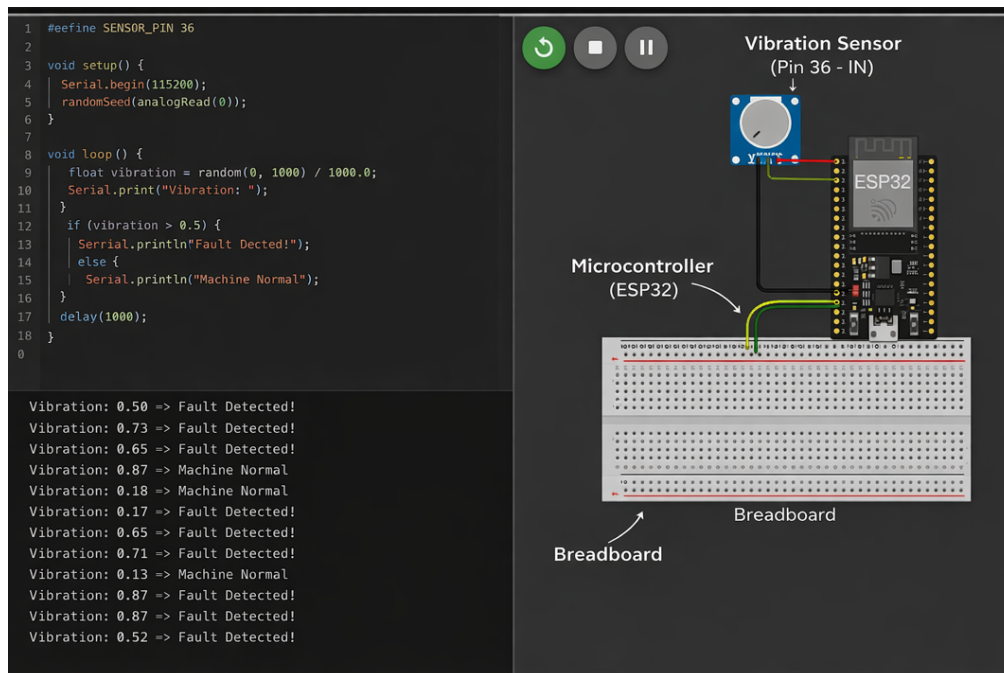
ML source code:

```
import tensorflow as tf  
import numpy as np  
# Sample dataset  
# Normal = small values, Fault = high values  
X = np.array([[0.1],[0.2],[0.15],[0.9],[1.0],[0.85]])  
y = np.array([0,0,0,1,1,1])  
# Model  
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(8, activation='relu', input_shape=(1,)),  
    tf.keras.layers.Dense(1, activation='sigmoid')  
)  
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])  
# Train  
model.fit(X, y, epochs=100)  
# Save model  
model.save("model.h5")
```

Output:

```
Vibration: 0.50 -> Fault Detected!  
Vibration: 0.73 -> Fault Detected!  
Vibration: 0.65 -> Fault Detected!  
Vibration: 0.87 -> Fault Detected!  
Vibration: 0.38 -> Machine Normal  
Vibration: 0.17 -> Machine Normal  
Vibration: 0.65 -> Fault Detected!  
Vibration: 0.71 -> Fault Detected!  
Vibration: 0.13 -> Machine Normal  
Vibration: 0.87 -> Fault Detected!  
Vibration: 0.52 -> Fault Detected!
```

Photos/ Screenshot/Snapshot :



Outcome:

The Smart Machine Fault Detection System successfully identifies whether a machine is operating normally or has a fault based on vibration data. It demonstrates the use of IoT and basic machine learning for predictive maintenance.

Dr . Aruna M G

Course Coordinator

Dr . Vindhya P Malagi

Head of the Department

Artificial Intelligence & Machine Learning

Dayananda Sagar College of Engineering

Bangalore-560111